Page Contents

Read This First(⇘ #support-notice-header)
About This Example(⇘ #aboutthisexample)
Examples(⇘ #examples)
Accessibility Features(⇘ #accessibilityfeatures)
Keyboard Support(⇘ #kbd_label)
Role, Property, State, and Tabindex Attributes(⇘ #rps_label)
JavaScript and CSS Source Code(⇘ #javascriptandcsssourcecode)
HTML Source Code(⇘ #htmlsourcecode)

Example Listboxes with Rearrangeable Options

Read This First

The code in this example is not intended for production environments. Before using it for any purpose, read this to understand why.

About This Example



The following two example implementations of the Listbox Pattern demonstrate differences between single-select and multi-select functionality. In both examples, users can use action buttons to move options from one list to another. In the first implementation, users can choose a single option and then activate an action button while in the second example, they may select multiple options before activating an action button.

Similar examples include:

- Scrollable Listbox Example: Single-select listbox that scrolls to reveal more options, similar to HTML `select` with `size` attribute greater than one.
- Listbox Example with Grouped Options: Single-select listbox with grouped options, similar to an HTML `select` with `optgroup` children.

Examples

Example 1: Single-Select Listbox

Open In CodePen

Rank features important to you when choosing where to live. If a feature is unimportant, move it to the unimportant features list.

Important Features:

- Proximity of public K-12 schools
- Proximity of child-friendly parks
- Proximity of grocery shopping
- Proximity of fast food
- Proximity of fine dining
- Neighborhood walkability
- Availability of public transit
- Proximity of hospital and medical services
- Level of traffic noise
- Access to major highways

Unimportant Features:

Up | Down | →Not Important | ←Important

Example 2: Multi-Select Listbox

Open In CodePen

Choose upgrades for your transport capsule.

Available upgrades:

- Leather seats
- Front seat warmers
- Rear bucket seats
- Rear seat warmers
- Front sun roof
- Rear sun roof
- Cloaking capability

Upgrades you have chosen:

Food synthesizer
Advanced waste recycling system
Turbo vertical take-off capability

→Add ←Remove

Accessibility Features

- Keyboard shortcuts for action buttons:
  - Action buttons have the following shortcuts:
    - "Up": `Alt` + `Up Arrow`
    - "Down": `Alt` + `Down Arrow`
    - "Add": `Enter`
    - "Not Important", "Important", and "Remove": `Delete`
  - Availability of the shortcuts is communicated to assistive technologies via the `aria-keyshortcuts` property on the button elements.
  - Each shortcut is only captured when focus is in a context where it is relevant. For example, `Enter` performs an add only when focus is in the available options list, and `Delete` performs a remove only when focus is in the chosen options list.
  - Using a shortcut key intentionally places focus to optimize both screen reader and keyboard usability. For example, pressing `Alt` + `Up Arrow` in the "Important Features" list keeps focus on the option that is moved up, enabling all keyboard users to easily perform consecutive move operations for an option and screen reader users to hear the position of an option after it is moved. Similarly, pressing `Enter` in the available options list leaves focus in the available options list. If the option that had focus before the add operation is no longer present in the list, focus lands on the first of the subsequent options that is still present.
- In example 1, since there are four action buttons, a toolbar widget is used to group all the action buttons into a single tab stop.
- Live regions provide confirmation of completed actions.
- Because this listbox implementation is scrollable and manages which option is focused by using aria-activedescendant, the JavaScript must ensure the focused option is visible. So, when a keyboard or pointer event changes the option referenced by `aria-activedescendant`, if the referenced option is not fully visible, the JavaScript scrolls the listbox to position the option in view.
- To enhance perceivability when operating the listbox, visual keyboard focus and hover are styled using the CSS `:hover` and `:focus` pseudo-classes:
  - To help people with visual impairments identify the listbox as an interactive element, the cursor is changed to a pointer when hovering over the list.

- To make it easier to distinguish the selected listbox option from other options, selection creates a 2 pixel border above and below the option.

Keyboard Support

The example listboxes on this page implement the following keyboard interface. Other variations and options for the keyboard interface are described in the Keyboard Interaction section of the Listbox Pattern.

NOTE: When visual focus is on an option in these implementations of listbox, DOM focus remains on the listbox element and the value of `aria-activedescendant` on the listbox refers to the descendant option that is visually indicated as focused. Where the following descriptions of keyboard commands mention focus, they are referring to the visual focus indicator, not DOM focus. For more information about this focus management technique, see Managing Focus in Composites Using aria-activedescendant.

Key	*Function*
Tab	• Moves focus into and out of the listbox. • When the listbox receives focus, if none of the options are selected, the first option receives focus. Otherwise, the first selected option receives focus.
Down Arrow	• Moves focus to the next option. • In the example 1 single-select listboxes, also selects the focused option.
Up Arrow	• Moves focus to the previous option. • In the example 1 single-select listboxes, also selects the focused option.
Home	• Moves focus to the first option. • In the example 1 single-select listboxes, also selects the focused option.
End	• Moves focus to the last option. • In the example 1 single-select listboxes, also selects the focused option.

Key	*Function*
Printable Characters	• Type a character: focus moves to the next item with a name that starts with the typed character. • Type multiple characters in rapid succession: focus moves to the next item with a name that starts with the string of characters typed.

Multiple selection keys supported in example 2

NOTE: The selection behavior demonstrated differs from the behavior provided by browsers for native HTML `<select multiple>` elements. The HTML select element behavior is to alter selection with unmodified up/down arrow keys, requiring the use of modifier keys to select multiple options. This example demonstrates the multiple selection interaction model recommended in the Keyboard Interaction section of the Listbox Pattern, which does not require the use of modifier keys.

Key	*Function*
Space	changes the selection state of the focused option .
Shift + *Down Arrow*	Moves focus to and selects the next option.
Shift + *Up Arrow*	Moves focus to and selects the previous option.
Control + *Shift* + *Home*	Selects from the focused option to the beginning of the list.
Control + *Shift* + *End*	Selects from the focused option to the end of the list.
Control + *A (All Platforms)* *Command–A (macOS)*	Selects all options in the list. If all options are selected, unselects all options.

Role, Property, State, and Tabindex Attributes

The example listboxes on this page implement the following ARIA roles, states, and properties. Information about other ways of applying ARIA roles, states, and properties is available in the Roles, States, and Properties section of the Listbox Pattern.

Role	Attribute	Element	Usage
`listbox`		`ul`	Identifies the focusable element that has listbox behaviors and contains the listbox options.
	`aria-labelledby="ID_REF"`	`ul`	Applied to the element with the listbox role, it refers to the span containing its label.
	`tabindex="0"`	`ul`	Applied to the element with the listbox role, it puts the listbox in the tab sequence.
	`aria-multiselectable="true"`	`ul`	• **Used only in example 2.** • Applied to the element with the listbox role, tells assistive technologies that the list supports multiple selection. • The default value is false so it is not specified in example 1.
	`aria-activedescendant="ID_REF"`	`ul`	• When an option in the listbox is visually indicated as having keyboard focus, refers to that option. • Enables assistive technologies to know which element the application regards as focused while DOM focus remains on the listbox element. • When navigation keys, such as `Down Arrow`, are pressed, the JavaScript changes the value. • When the listbox is empty, `aria-activedescendant=""`. • For more information about this focus management technique, see Managing

Role	Attribute	Element	Usage
			Focus in Composites Using aria-activedescendant.
option		`li`	Identifies each selectable element containing the name of an option.
	aria-selected="true"	`li`	• Applied to elements with role option that are visually styled as selected to inform assistive technologies that the options are selected. • In example 1, the option with this attribute is always the same as the option that is referenced by aria-activedescendant because it is a single-select listbox where selection follows focus. • In example 2, more than one option may be selected so the user can move focus among options without effecting which options have this attribute. • Note that in example 2, the focus style and selection styles are distinctly different and independent.

Role	Attribute	Element	Usage
	aria-selected="false"	li	• **Used only in example 2.** • Applied to elements with role option that are **not** visually styled as selected to inform assistive technologies that the options are selectable but not selected. • In example 1, this is unnecessary because the selection moves every time the focus moves.
	aria-hidden="true"	span	Removes the character entities used for the check mark, left arrow and right arrow from the accessibility tree to prevent them from being included in the accessible name of an option or button.

JavaScript and CSS Source Code

- CSS: listbox.css
- JavaScript: listbox.js, toolbar.js, listbox-rearrangeable.js

HTML Source Code

Example 1: Single-Select Listbox

To copy the following HTML code, please open it in CodePen.

Open In CodePen

```
<p>
  Rank features important to you when choosing where to live.
  If a feature is unimportant, move it to the unimportant features list.
</p>
<div class="listbox-area">
```

```
<div class="left-area">
  <span id="ss_imp_l" class="listbox-label">
    Important Features:
  </span>
  <ul id="ss_imp_list"
      tabindex="0"
      role="listbox"
      aria-labelledby="ss_imp_l">
    <li id="ss_opt1" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Proximity of public K-12 schools
    </li>
    <li id="ss_opt2" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Proximity of child-friendly parks
    </li>
    <li id="ss_opt3" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Proximity of grocery shopping
    </li>
    <li id="ss_opt4" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Proximity of fast food
    </li>
    <li id="ss_opt5" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Proximity of fine dining
    </li>
    <li id="ss_opt6" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Neighborhood walkability
    </li>
    <li id="ss_opt7" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Availability of public transit
    </li>
    <li id="ss_opt8" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Proximity of hospital and medical services
    </li>
    <li id="ss_opt9" role="option">
      <span class="checkmark" aria-hidden="true"></span>
      Level of traffic noise
    </li>
    <li id="ss_opt10" role="option">
      <span class="checkmark" aria-hidden="true"></span>
```

```
      Access to major highways
    </li>
  </ul>
  <div role="toolbar"
       aria-label="Actions"
       class="toolbar">
    <button type="button"
            id="ex1-up"
            class="toolbar-item selected"
            aria-keyshortcuts="Alt+ArrowUp"
            aria-disabled="true">
      Up
    </button>
    <button type="button"
            id="ex1-down"
            class="toolbar-item"
            tabindex="-1"
            aria-keyshortcuts="Alt+ArrowDown"
            aria-disabled="true">
      Down
    </button>
    <button type="button"
            id="ex1-delete"
            class="toolbar-item move-right-btn"
            tabindex="-1"
            aria-keyshortcuts="Alt+ArrowRight Delete"
            aria-disabled="true">
      <span class="checkmark" aria-hidden="true"></span>
      Not Important
    </button>
  </div>
</div>
<div class="right-area">
  <span id="ss_unimp_l" class="listbox-label">
    Unimportant Features:
  </span>
  <ul id="ss_unimp_list"
      tabindex="0"
      role="listbox"
      aria-labelledby="ss_unimp_l"
      aria-activedescendant=""></ul>
  <button type="button"
          id="ex1-add"
          class="move-left-btn"
          aria-keyshortcuts="Alt+ArrowLeft Enter"
          aria-disabled="true">
```

```
      <span class="checkmark" aria-hidden="true"></span>
      Important
    </button>
  </div>
  <div class="offscreen">
    Last change:
    <span aria-live="polite" id="ss_live_region"></span>
  </div>
</div>
```

Example 2: Multi-Select Listbox

To copy the following HTML code, please open it in CodePen.

Open In CodePen

```
<p>
  Choose upgrades for your transport capsule.
</p>
<div class="listbox-area">
  <div class="left-area">
    <span id="ms_av_l" class="listbox-label">
      Available upgrades:
    </span>
    <ul id="ms_imp_list"
        tabindex="0"
        role="listbox"
        aria-labelledby="ms_av_l"
        aria-multiselectable="true">
      <li id="ms_opt1"
          role="option"
          aria-selected="false">
        <span class="checkmark" aria-hidden="true"></span>
        Leather seats
      </li>
      <li id="ms_opt2"
          role="option"
          aria-selected="false">
        <span class="checkmark" aria-hidden="true"></span>
        Front seat warmers
      </li>
      <li id="ms_opt3"
          role="option"
          aria-selected="false">
```

```
      <span class="checkmark" aria-hidden="true"></span>
      Rear bucket seats
    </li>
    <li id="ms_opt4"
        role="option"
        aria-selected="false">
      <span class="checkmark" aria-hidden="true"></span>
      Rear seat warmers
    </li>
    <li id="ms_opt5"
        role="option"
        aria-selected="false">
      <span class="checkmark" aria-hidden="true"></span>
      Front sun roof
    </li>
    <li id="ms_opt6"
        role="option"
        aria-selected="false">
      <span class="checkmark" aria-hidden="true"></span>
      Rear sun roof
    </li>
    <li id="ms_opt7"
        role="option"
        aria-selected="false">
      <span class="checkmark" aria-hidden="true"></span>
      Cloaking capability
    </li>
    <li id="ms_opt8"
        role="option"
        aria-selected="false">
      <span class="checkmark" aria-hidden="true"></span>
      Food synthesizer
    </li>
    <li id="ms_opt9"
        role="option"
        aria-selected="false">
      <span class="checkmark" aria-hidden="true"></span>
      Advanced waste recycling system
    </li>
    <li id="ms_opt10"
        role="option"
        aria-selected="false">
      <span class="checkmark" aria-hidden="true"></span>
      Turbo vertical take-off capability
    </li>
  </ul>
```

```
    <button type="button"
            id="ex2-add"
            class="move-right-btn"
            aria-keyshortcuts="Alt+ArrowRight Enter"
            aria-disabled="true">
      <span class="checkmark" aria-hidden="true"></span>
      Add
    </button>
  </div>
  <div class="right-area">
    <span id="ms_ch_l" class="listbox-label">
      Upgrades you have chosen:
    </span>
    <ul id="ms_unimp_list"
        tabindex="0"
        role="listbox"
        aria-labelledby="ms_ch_l"
        aria-activedescendant=""
        aria-multiselectable="true">
    </ul>
    <button type="button"
            id="ex2-delete"
            class="move-left-btn"
            aria-keyshortcuts="Alt+ArrowLeft Delete"
            aria-disabled="true">
      <span class="checkmark" aria-hidden="true"></span>
      Remove
    </button>
  </div>
  <div class="offscreen">
    Last change:
    <span aria-live="polite" id="ms_live_region"></span>
  </div>
</div>
```

View issues related to this example (https://github.com/orgs/w3c/projects/131)

Page last updated: 12 August 2025

W3C Web Accessibility Initiative (WAI) (https://www.w3.org/WAI/)

Strategies, standards, resources to make the Web accessible to people with disabilities

Get News in Email (https://www.w3.org/WAI/news/subscribe/)

LinkedIn (https://www.linkedin.com/company/w3c-wai/) Mastodon (https://w3c.social/@wai)

Copyright © 2026 World Wide Web Consortium (https://www.w3.org/).
W3C® liability (https://www.w3.org/policies/#disclaimers), trademark (https://www.w3.org/policies/#trademarks) and permissive license (https://www.w3.org/copyright/software-license/) rules apply unless otherwise noted. See Permission to Use WAI Material (https://www.w3.org/WAI/about/using-wai-material/).